Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Макеев И.М.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 02.10.26

Москва, 2026

Постановка задачи

Вариант 20. Родительский процесс создает два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1 или в pipe2 в зависимости от правила фильтрации. Процессы child1 и child2 производят работу над строками. Процессы пишут результаты своей работы в стандартный вывод. Правило фильтрации: строки длины больше 10 символов отправляются в pipe2, иначе в pipe1. Дочерние процессы инвертируют строки. Программа должна быть написана на языке Си, при этом необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Общий метод и алгоритм решения

Использованные интерфейсы ОС: fork - создание процесса; pipe - неименованные каналы; dup2 - переназначение дескрипторов; open - открытие выходного файла с O_WRONLY | O_CREAT | O_TRUNC и правами 0644 (с учетом umask); close - закрытие дескрипторов; read и write - чтение и запись; waitpid - ожидание потомков. execl запускает ./child через семейство exec; signal задает игнорирование SIGPIPE. exit - библиотечная функция завершения процесса, а не отдельный системный вызов.

Реализованы две программы: parent.c и child.c; общие функции ввода-вывода вынесены в io_utils.h. Родитель читает два имени выходных файлов, создает pipe1 и pipe2 и запускает двух потомков через fork.

В каждом потомке открывается свой файл, dup2 перенаправляет стандартный ввод на читающий конец своего канала, а стандартный вывод - в файл. Исходные дескрипторы файла и обоих каналов закрываются. Дескрипторы 0 и 1 сохраняют нужные ссылки. Затем execl заменяет образ процесса программой child.

Родитель закрывает читающие концы каналов и получает строки через read_line. Длина определяется strcspn до символа перевода строки. При n > 10 строка отправляется в pipe2, иначе в pipe1. write_all дописывает данные после частичной записи и повторяет write при EINTR. Перед отправкой добавляется перевод строки, в том числе для последней строки без него.

Дочерняя программа читает строки через read_line, убирает перевод строки, меняет местами симметричные байты и записывает результат и перевод строки через write_all в стандартный вывод, уже направленный в файл.

При окончании ввода родитель закрывает записывающие концы каналов. Когда все ссылки на запись закрыты, потомки дочитывают канал и получают EOF. Родитель вызывает waitpid для каждого созданного потомка и проверяет WIFEXITED и WEXITSTATUS.

io_utils.h содержит read_line (побайтовое read и расширение буфера realloc), write_all (полная запись) и print_error (сообщение в дескриптор 2 через write_all, текст причины через strerror). read_line и write_all повторяют прерванные операции при EINTR.

Проверяются ошибки pipe, fork, open, dup2, execl, рабочих записей и waitpid. SIGPIPE игнорируется, поэтому запись в канал без читателя возвращает ошибку EPIPE. Полнота обработки ошибок ограничена: EOF и ошибка read возвращаются как -1; ошибки чтения могут завершать цикл без диагностики и с кодом 0. Результаты signal, close и записей приглашений не проверяются. waitpid не повторяется при EINTR.

Код программы

Ниже приведены полные исходники из main на момент проверки 31.10.2026, коммит 92173346e75c4271d4fa40357386aa85835fddc0. Код приведен без исправлений, чтобы отчет соответствовал репозиторию.

parent.c

```
#define _POSIX_C_SOURCE 200809L
#include <fcntl.h>
#include <signal.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>
#include "io_utils.h"
pid_t start_child(const char *file, int in[2], int other[2]) {
    pid_t pid = fork();
    if (pid == -1) {
        print_error("fork");
        return -1;
    }
    if (pid == 0) {
        int out = open(file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (out == -1) {
            print_error("open");
            exit(1);
        }
        if (dup2(in[0], STDIN_FILENO) == -1 || dup2(out, STDOUT_FILENO) == -1) {
            print_error("dup2");
            exit(1);
        }
        close(out);
        close(in[0]);
        close(in[1]);
        close(other[0]);
        close(other[1]);
        execl("./child", "child", NULL);
        print_error("execl");
        exit(1);
    }
    return pid;
}
int main(void) {
    signal(SIGPIPE, SIG_IGN);
    char *line = NULL;
    size_t cap = 0;
    write_all(STDOUT_FILENO, "Файл для child1: ", sizeof("Файл для child1: ") - 1);
    if (read_line(STDIN_FILENO, &line, &cap) == -1) return 1;
    line[strcspn(line, "\n")] = '\0';
    if (line[0] == '\0') {
       write_all(STDERR_FILENO, "Имя файла не введено\n", sizeof("Имя файла не введено\n") - 1);
        return 1;
    }
    char *file1 = strdup(line);
    write_all(STDOUT_FILENO, "Файл для child2: ", sizeof("Файл для child2: ") - 1);
    if (read_line(STDIN_FILENO, &line, &cap) == -1) return 1;
    line[strcspn(line, "\n")] = '\0';
    if (line[0] == '\0') {
       write_all(STDERR_FILENO, "Имя файла не введено\n", sizeof("Имя файла не введено\n") - 1);
        return 1;
    }
    char *file2 = strdup(line);
    if (file1 == NULL || file2 == NULL) {
        print_error("strdup");
        return 1;
    }
    int pipe1[2], pipe2[2];
    if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
        print_error("pipe");
        return 1;
    }
    pid_t pid1 = start_child(file1, pipe1, pipe2);
    pid_t pid2 = -1;
    if (pid1 != -1) pid2 = start_child(file2, pipe2, pipe1);
    close(pipe1[0]);
    close(pipe2[0]);
    int result = 0;
```

```
    if (pid1 == -1 || pid2 == -1) {
        result = 1;
    } else {
      write_all(STDOUT_FILENO, "Вводите строки (Ctrl+D - конец):\n", sizeof("Вводите строки (Ctrl+D - конец):\n") - 1);
        while (read_line(STDIN_FILENO, &line, &cap) != -1) {
            size_t n = strcspn(line, "\n");
            line[n] = '\n';
            int fd = (n > 10) ? pipe2[1] : pipe1[1];
            if (write_all(fd, line, n + 1) == -1) {
                print_error("write");
                result = 1;
                break;
            }
        }
    }
    close(pipe1[1]);
    close(pipe2[1]);
    int status;
    if (pid1 > 0) {
       if (waitpid(pid1, &status, 0) == -1 || !WIFEXITED(status) || WEXITSTATUS(status) != 0) result = 1;
    }
    if (pid2 > 0) {
       if (waitpid(pid2, &status, 0) == -1 || !WIFEXITED(status) || WEXITSTATUS(status) != 0) result = 1;
    }
    free(file1);
    free(file2);
    free(line);
    return result;
}
```

child.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdlib.h>
#include <string.h>
#include "io_utils.h"
int main(void) {
    char *line = NULL;
    size_t cap = 0;
    while (read_line(STDIN_FILENO, &line, &cap) != -1) {
        size_t n = strcspn(line, "\n");
        line[n] = '\0';
        for (size_t i = 0; i < n / 2; i++) {
            char t = line[i];
            line[i] = line[n - 1 - i];
            line[n - 1 - i] = t;
        }
        if (write_all(STDOUT_FILENO, line, n) == -1 ||
            write_all(STDOUT_FILENO, "\n", 1) == -1) {
            print_error("write");
            free(line);
            return 1;
        }
    }
    free(line);
    return 0;
}
```

io_utils.h

```
#ifndef IO_UTILS_H
#define IO_UTILS_H
#include <errno.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <unistd.h>
static ssize_t read_line(int fd, char **line, size_t *cap) {
    size_t len = 0;
    for (;;) {
        char ch;
        ssize_t n = read(fd, &ch, 1);
        if (n == 0) {
            if (len == 0) return -1;
            break;
        }
        if (n == -1) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (len + 2 > *cap) {
            size_t new_cap = *cap ? *cap * 2 : 128;
            if (new_cap < len + 2) new_cap = len + 2;
            char *p = realloc(*line, new_cap);
            if (p == NULL) return -1;
            *line = p;
            *cap = new_cap;
        }
        (*line)[len++] = ch;
        if (ch == '\n') break;
    }
    (*line)[len] = '\0';
    return (ssize_t)len;
}
static int write_all(int fd, const void *buffer, size_t length) {
    const char *p = buffer;
    while (length > 0) {
        ssize_t n = write(fd, p, length);
        if (n == -1) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (n == 0) {
            errno = EIO;
            return -1;
        }
        p += n;
        length -= (size_t)n;
    }
    return 0;
}
static void print_error(const char *operation) {
    int saved_errno = errno;
    write_all(STDERR_FILENO, operation, strlen(operation));
    write_all(STDERR_FILENO, ": ", 2);
    const char *description = strerror(saved_errno);
    write_all(STDERR_FILENO, description, strlen(description));
    write_all(STDERR_FILENO, "\n", 1);
}
#endif
```

Протокол работы программы

Проверка выполнена 31.10.2026 в macOS с компилятором Apple clang (cc). Сборка обеих программ: cc -std=c11 -Wall -Wextra -Wpedantic -Werror -o parent parent.c; аналогично для child.c. Предупреждений нет. Запуск выполнялся из каталога с исполняемым файлом child, поскольку используется путь ./child.

Тест 1. Компиляция и обычная работа

```
$ ./parent
Файл для child1: out1.txt
Файл для child2: out2.txt
Вводите строки (Ctrl+D - конец):
hello
abc
this is a long line
0123456789
01234567890
[Ctrl+D]
$ cat out1.txt
olleh
cba
9876543210
$ cat out2.txt
enil gnol a si siht
09876543210
```

Фактические файлы совпали с ожидаемыми. Строки длиной 10 байт попадают в pipe1, длиной 11 байт - в pipe2.

Тест 2. Ошибка открытия файла

Для первого файла /nonexistent/x.txt, второго out2.txt и окончания ввода получено сообщение open: No such file or directory; код родителя 1. Порядок сообщения относительно приглашения может отличаться из-за параллельного выполнения процессов.

Дополнительные проверки

Успешно проверены пустые строки, пустой поток данных, последняя строка без перевода строки и строка из 200 000 ASCII-байтов. Содержимое обоих файлов сравнивалось побайтово с ожидаемым; код родителя во всех этих случаях 0.

Отрицательная проверка: child запущен со стандартным вводом, открытым только для записи. read получает EBADF, однако child возвращает 0 и не выводит сообщение. Это подтвержденный дефект обработки ошибок, а не успешный тест.

Соответствие ограничениям и замечания

Ограничение на ввод-вывод без stdio выполнено: в трех файлах отсутствуют stdio.h, FILE, getline, printf, fprintf, fflush и perror. Ввод-вывод реализован через read/write и собственные обертки.

Буквальное требование «только системные вызовы» не выполнено: используются realloc, free, strdup, strlen, strcspn, strerror, exit и библиотечные POSIX-интерфейсы. Если запрещен именно библиотечный ввод-вывод, это не является нарушением. Текст методических указаний в репозитории отсутствует, поэтому более строгую трактовку подтвердить нельзя.

Длина и инверсия считаются по байтам, а не по символам Unicode. Для ASCII результат корректен; UTF-8-кириллица может уйти в неверный канал по числу символов, а переворот байтов повреждает кодировку. Встроенный нулевой байт обрезает обработку через strcspn. Вход фактически должен быть текстом без NUL в однобайтовой кодировке.

Рекомендуется различать EOF и ошибку чтения, проверять все важные результаты, повторять waitpid при EINTR и использовать _exit в ветках ошибок после fork. Нужна защита от переполнения размера при расширении буфера. Ранние возвраты не всегда освобождают уже выделенную память; при ошибке второго pipe первый остается открытым до завершения процесса.

Выходные файлы открываются с усечением. Если задать один файл (или два пути к одному файлу) обоим потомкам, результат может быть перезаписан или смешан; для штатной работы нужны разные файлы. Код также предполагает, что стандартные дескрипторы 0, 1 и 2 открыты при запуске.

README устарел: в нем еще указана обработка через perror и не перечислен io_utils.h. Эти сведения следует привести в соответствие с кодом.

Вывод

В ходе лабораторной работы были изучены создание процессов через fork и замена их образа через execl, передача данных между процессами через неименованные каналы и переназначение стандартных потоков ввода-вывода с помощью dup2. Реализована программа, в которой родитель распределяет строки между двумя дочерними процессами по длине, а потомки инвертируют их и записывают в файлы. Основное внимание при отладке потребовалось уделить закрытию неиспользуемых концов каналов: пока остается открытым лишний записывающий конец, потомок не получает конец файла и не завершается. Реализована обработка ряда системных ошибок; ограничения текущей реализации перечислены выше.